

```
// このクラスが入っているパッケージ名です
package jp.co.active.kenshu.login;

// 入出力エラーを扱うために使います
import java.io.IOException;

// フィルターを作るために必要です
import jakarta.servlet.Filter;

// 次のフィルターやServletへ処理を進めるために必要です
import jakarta.servlet.FilterChain;

// Servlet関係の例外を扱うために必要です
import jakarta.servlet.ServletException;

// リクエスト情報を扱うために必要です
import jakarta.servlet.HttpServletRequest;

// レスpons情報を扱うために必要です
import jakarta.servlet.HttpServletResponse;

// アノテーションでURLを指定するために必要です
import jakarta.servlet.annotation.WebFilter;

// HTTP用のリクエスト情報を使うために必要です
import jakarta.servlet.http.HttpServletRequest;

// HTTP用のレスpons情報を使うために必要です
import jakarta.servlet.http.HttpServletResponse;

// セッションを使うために必要です
import jakarta.servlet.http.HttpSession;

// /calendar/ から始まるURLにアクセスされたとき、このフィルターを通します
@WebFilter("/calendar/*")

// ログインチェックを行うフィルターです
public class LoginCheckFilter implements Filter {

    // /calendar/ から始まるURLにアクセスされたときに実行されます
```

@Override

public void doFilter(

 ServletRequest request,

 ServletResponse response,

 FilterChain chain

) throws IOException, ServletException {

 // ServletRequest を HttpServletRequest に変換します

 // セッションや URL を扱うために必要です

 HttpServletRequest req = (HttpServletRequest) request;

 // ServletResponse を HttpServletResponse に変換します

 // リダイレクトを使うために必要です

 HttpServletResponse resp = (HttpServletResponse) response;

 // アプリケーションの基本パスを取得します

 String contextPath = req.getContextPath();

 // アクセスされた URL 全体を取得します

 String uri = req.getRequestURI();

 // contextPath を除いた実際のパスを取り出します

 String path = uri.substring(contextPath.length());

 // ログイン画面はログイン前にアクセスする必要があるので除外します

 // ログイン処理もログイン前に実行する必要があるので除外します

 // ログアウト処理も通すために除外します

 if (path.equals("/calendar/login"))

 || path.equals("/calendar/login/process")

 || path.equals("/calendar/logout")) {

 // チェックせずに次の処理へ進めます

 chain.doFilter(request, response);

 // ここで処理を終了します

 return;

}

 // すでに存在しているセッションを取得します

 // false なので、セッションがない場合は新しく作りません

 HttpSession session = req.getSession(false);

```
// セッションがない、またはisLoginがtrueでない場合は未ログインです
if (session == null || !Boolean.TRUE.equals(session.getAttribute("isLogin"))) {

    // 未ログインの場合はログイン画面へ戻します
    resp.sendRedirect(contextPath + "/calendar/login");

    // ここで処理を終了します
    return;
}

// ブラウザにログイン後の画面を保存させないための設定です
resp.setHeader("Cache-Control", "no-store, no-cache, must-revalidate");

// 古いブラウザ向けにキャッシュを禁止する設定です
resp.setHeader("Pragma", "no-cache");

// ページの有効期限を0にして、保存済みページを使わせない設定です
resp.setDateHeader("Expires", 0);

// ログイン済みなら、目的のServletやJSPへ処理を進めます
chain.doFilter(request, response);
}
}
```